
Algoritmos y Estructuras de Datos

Tema 1: Introducción, Repaso de programación

Algoritmos y Estructuras de datos DIT-UPM

1

● 1

Contenidos

1. Repaso de herramientas

2. Repaso de programación

2.1 Tipos simples de datos y expresiones

2.2 Sentencias

2.3 Clases y Objetos *clases empiezan con mayuscula*

2.4 Interfaces y Herencia

2.5 Excepciones

2.6 Bibliotecas

Algoritmos y Estructuras de datos DIT-UPM

2

● 2

Enteros

- Hay cuatro tipos primitivos para representar enteros: **int**, **long**, **short** y **byte**
- Los literales son los mismos para todos ellos:
... -3, -2, -1, 0, 1, 2, 3 ...

Entero	Bits	Mínimo	Máximo
int	32	Integer.MIN_VALUE -2^{31} (- 2147483658)	Integer.MAX_VALUE $2^{31}-1$ (2147483657)
short	16	Short.MIN_VALUE -2^{15} (-32768)	Short.MAX_VALUE $2^{15}-1$ (32767)
long	64	Long.MIN_VALUE -2^{63} (-9223372036854775808)	Long.MAX_VALUE $2^{63}-1$ (9223372036854775807)
byte	8	Byte.MIN_VALUE -2^7 (-128)	Byte.MAX_VALUE 2^7-1 (127)

Algoritmos y Estructuras de datos DIT-UPM

●3

Operadores de Enteros

- Los operadores de los enteros son:
 - Unarios aritméticos (preceden a la expresión):
 - - invierte signo del entero
 - Binarios aritméticos (operan sobre sus operandos a izquierda y derecha):
 - +, - Suma y resta
 - * Multiplicación
 - / división que devuelve enteros $6 / 7 == 0$, $15 / 4 == 3$
 - % módulo (el resto cuando los números son positivos)
 $x \% y == x - (x / y) * y$
 - ++, -- pre/post incremento/decremento
 - Binarios relacionales (comparan valores de las expresiones a izquierda y derecha):
 - >, >=, <, <= Menor, menor o igual, mayor, mayor o igual
 - ==, != igual, distinto

Algoritmos y Estructuras de datos DIT-UPM

●4

Reales

- Número reales en coma flotante (mantisa y exponente): **double**, **float**
- Número limitado de cifras significativas y de exponente

Real	Bits	Mínimo Menor Constante Representable	Máximo Mayor Constante Representable
float	32	Float.MIN_VALUE	Float.MAX_VALUE
double	64	Double.MIN_VALUE	Double.MAX_VALUE

double > float

Algoritmos y Estructuras de datos DIT-UPM

●5

`float array[] = {1.5f, 2.5f};` //HAY QUE PONER LA f PARA QUE SEA FLOAT

Operadores de Reales

- Los operadores son los mismos que los enteros pero admiten/devuelven reales
 - Unarios aritméticos: -
 - Binarios aritméticos: +, -, *, /
 - Binarios relacionales: >, >=, <, <=, ==, !=
 - 0.0 == 0.0 es true, pero en general == y != nos puede dar resultados no esperados
 - Por ejemplo, prueba en scapbook que da false!:
`float x = 1.1F; double y = 1.1D;`
`double z = 0.0D; boolean cero;`
`cero = (x - y) == z; // cero pasa a valer false`
`cero = 0.0 == 0.0; // cero pasa a valer true`

Algoritmos y Estructuras de datos DIT-UPM

●6

Lógicos

- Tipo primitivo para valores lógicos: **boolean**
- Los literales: **true**, **false**
- Operadores sobre booleanos (son el resultado de operadores relacionales):
 - Unarios: ! negación
 - Binarios:
 - **x && y**: para que sea true, x e y deben ser true
 - Si x es false y no se evalúa
 - **x || y**: será true si cualquiera de las dos (x o y) es true
 - Si x es true y no se evalúa

Algoritmos y Estructuras de datos DIT-UPM

● 7

Ejemplos PrimitivosEnteros

Algoritmos y Estructuras de datos DIT-UPM

● 8

Arrays

Agrupación ordenada de longitud fija de datos de un mismo tipo:

- Declaración: `double[] v; MiClase w[];`
- Creación con constructor: `v = new double[14]; x = new MiClase[10];`
- Creación con literal: `v = {5.0, 5E0, 5D}; w = {new MiClase()};`
- Acceso: `v[3] = 7D; double d = v[3];`

tipo [] nombre = new tipo[tamaño]

Dimensiones:

- length: Almacena el tamaño del array -> `v.length`
- Las posiciones de un array van de 0 a n-1

Multidimensión:

- Se pueden definir array de varias dimensiones
- Cada posición del array almacena otro array

`-int[][] matriz= {{ 1, 2, 3 }, { 4, 5, 6 }};`

*[1,2,3] 1er array
[4,5,6] contiene 2 arrays*

Comparar arrays:

- Para comparar dos arrays hay que comparar si todos sus elementos son iguales y están en el mismo orden -> `a1 == a2`

Algoritmos y Estructuras de Datos DIT-UPM

9

● 9

Ejemplos Arrays

Algoritmos y Estructuras de datos DIT-UPM

● 10

Cadenas de Caracteres

String es una clase que representa cadenas de caracteres inmutables

Creación:

- Usando uno de los constructores disponibles
- A partir de un literal: "Ola ke ase!"

Manejo:

- Métodos de la clase String
- El operador + para concatenar cadenas
- Otras clases de Java:
 - `java.lang.StringBuffer`, `java.lang.StringBuilder`
 - `java.util.StringTokenizer`
 - `java.util.Regex.*`

Algoritmos y Estructuras de Datos DIT-UPM

11

● 11

Ejemplos Strings

Algoritmos y Estructuras de datos DIT-UPM

● 12

Clases envoltorio

Los tipos primitivos no son objetos

-Existen **clases envoltorio** que permiten su representación como objetos (java.lang.*): Integer, Long, Character, Boolean, Float, Double...

Conversión tipo primitivo → Objeto:

- Constructor: `Integer objInt = new Integer(i);`
- Método valueOf: `Integer objInt = Integer.valueOf(i);`
- Autoboxing

Conversión Objeto → Tipo primitivo:

- Método intValue: `int i = objInt.intValue();`
- Unboxing

tipo primitivo nombre = nombre.intValue();

Conversión tipo primitivo → String:

- String s = "" + i;

Conversión String → Tipo primitivo:

- int i = Integer.parseInt(s);

Algoritmos y Estructuras de Datos DIT-UPM

13

● 13

Objeto { Clase nombre = new Clase (atributos);

Contenidos

1. Repaso de herramientas

2. Repaso de programación

2.1 Tipos simples de datos y expresiones

2.2 Sentencias

2.3 Clases y Objetos

2.4 Interfaces y Herencia

2.5 Excepciones

2.6 Bibliotecas

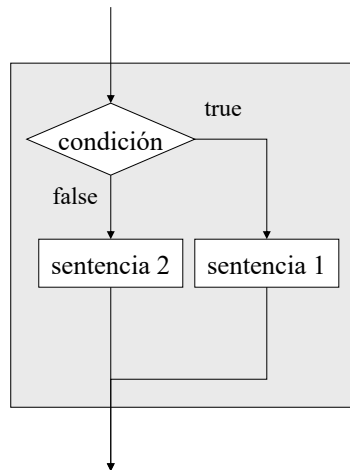
3. Control de Versiones

Algoritmos y Estructuras de datos DIT-UPM

14

● 14

Ejecución Condicional



```
if (hora < 14)
    System.out.println ("buenos dias");
else
    System.out.println ("buenas tardes");
```

```
if (condicion) sentencia1;
else sentencia2;
```

```
if (condicion) {
    sentencia1;
    sentencia2;
} else {
    sentencia3;
    sentencia4;
}
```

Algoritmos y Estructuras de datos DIT-UPM

● 15

Ejemplos Condicional2

Algoritmos y Estructuras de datos DIT-UPM

● 16

Selección múltiple

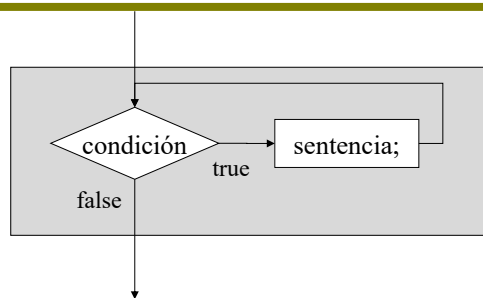
```
switch (verbosidad) {  
  case EXTREMA:  
    System.out.println(todosLosDetalles);  
  case NOTIFICACIONES:  
    System.out.println(lasNotificaciones);  
  case ERRORES:  
    System.out.println(losErrores);  
    break;  
  default:  
    System.out.println("nivel de verbosidad desconocida");  
}
```

Algoritmos y Estructuras de datos DIT-UPM

● 17

```
switch (tipoInt) {  
  case 18:  
  case 15:  
}
```

while



```
while (condicion)  
  sentenciaSimple;  
  
while(condicion) {  
  bloque sentencias;  
}
```

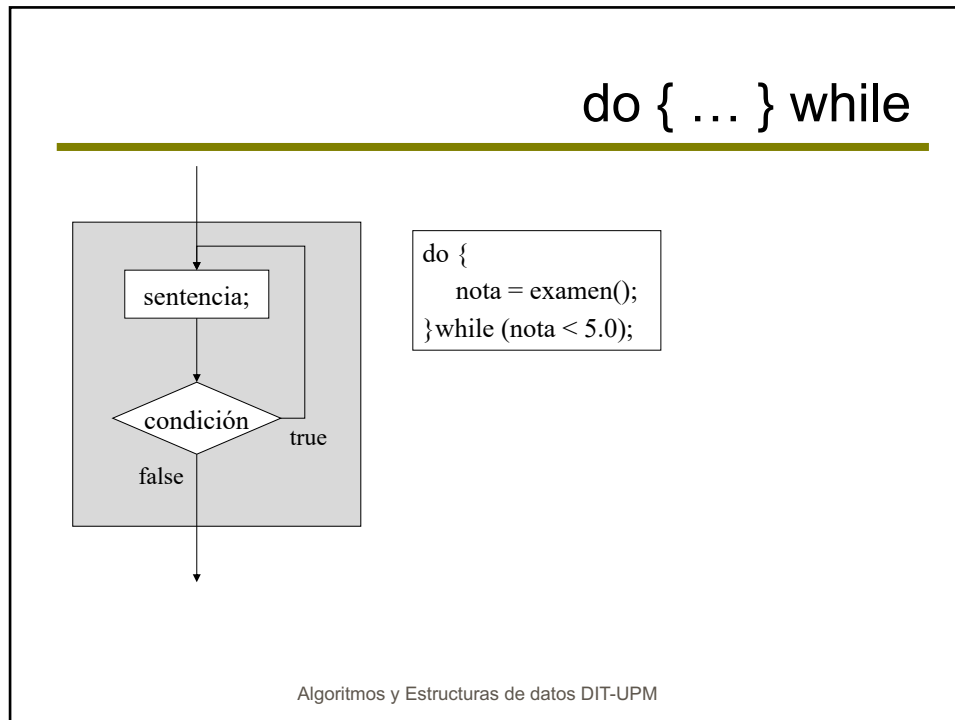
```
while (!aprobado)  
  me.examino();
```

```
while (llueve())  
  uso.paraguas();
```

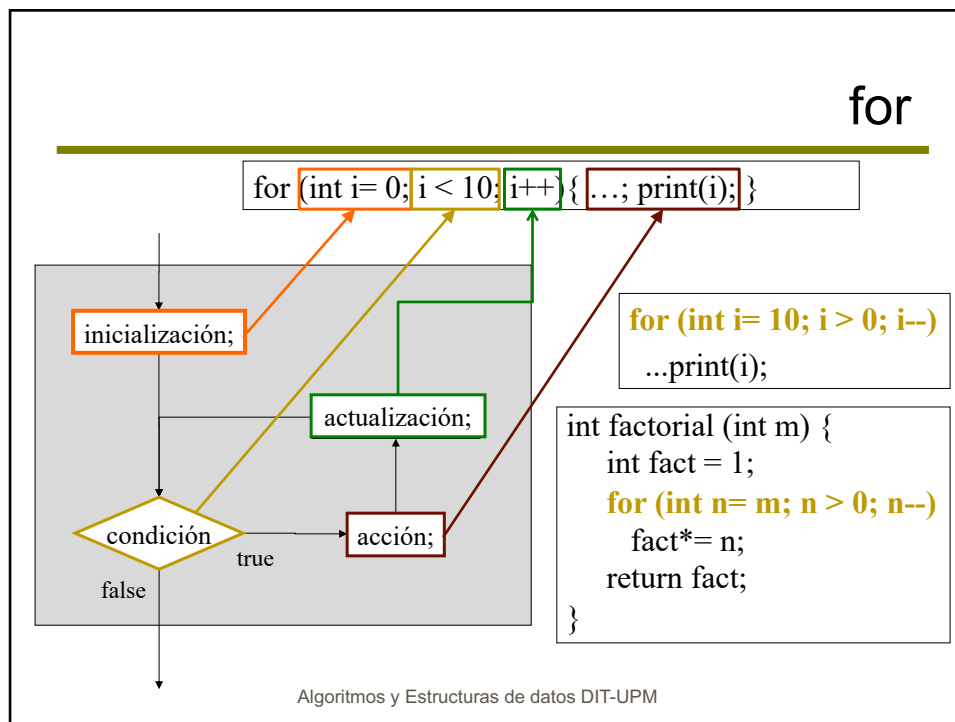
Algoritmos y Estructuras de datos DIT-UPM

● 18

while evalua la condicion antes de ejecutar el bloque de código. Do while ejecuta el bloque al menos una vez



● 19



● 20

Ejemplos Piramide

Algoritmos y Estructuras de datos DIT-UPM

● 21

Ejercicios

1. Crear un array de 100 enteros y rellenarlo con el cuadrado de cada posición: [0, 1, 4, 9...]
2. Dado un array de enteros, encontrar el valor mínimo almacenado
3. Dado un array de enteros, encontrar la posición del valor mínimo
4. Dado un array de enteros, calcular la suma de sus valores
5. Dado un array de enteros, invertir sus términos:
 - a) Generando un nuevo array
 - b) Reordenando los términos en el array existente
6. Dado un array de enteros, calcular el valor medio de los valores almacenados
7. Dado un String, devolver un array de cinco posiciones que cuente las veces que aparece cada vocal
8. Dado un array de enteros, calcular el factorial de cada uno

Algoritmos y Estructuras de Datos DIT-UPM

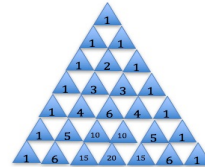
22

● 22

Ejercicios

El triángulo de Pascal es una herramienta de cálculo algebraico utilizado en combinatoria y cálculo algebraico en general. Se construye de la siguiente forma: partimos de una única hilera de un solo número con valor 1, debajo escribimos una segunda hilera con dos 1. Cada una de las siguientes hileras tiene un número más y comienzan y terminan por 1, cada número intermedio se calcula como la suma de los dos que tiene encima en la hilera de anterior. Tenemos la clase TrianguloPascal que construye el triángulo de Pascal.

```
public class TrianguloPascal {
    private int[][] tp;
    public TrianguloPascal(int filas) {
        tp = new int[filas][];
        for (int i=0; i < filas; i++) {
            tp[i] = new int[i+1];
            tp[i][0] = 1;
            tp[i][i] = 1;
            for (int j=1; j < i; j++)
                tp[i][j] = tp[i-1][j-1] + tp[i-1][j];
        }
    }
}
```



El triángulo de Pascal permite leer directamente combinaciones de números. Por ejemplo, la fila 1, 5, 10, 10, 5, 1, enumera las formas de seleccionar objetos de un grupo de 5. Hay 5 formas de seleccionar un elemento de 5; 10 maneras de seleccionar 2, diez maneras de seleccionar 3; 5 de seleccionar 4 y solo 1 de seleccionar las 5.

Se pide: tomando como base el triángulo de Pascal escribir un método en la clase TrianguloPascal que reciba dos parámetros, el número de elementos del grupo y el número de elementos que se toman, devolverá el número de combinaciones en que se pueden tomar (por ejemplo, si de un grupo de 5 tomamos 3, debemos devolver 10, el número de posibles combinaciones). El método debe asegurarse que los parámetros son correctos y levantar excepción si no lo son.

● 23

Ejercicios

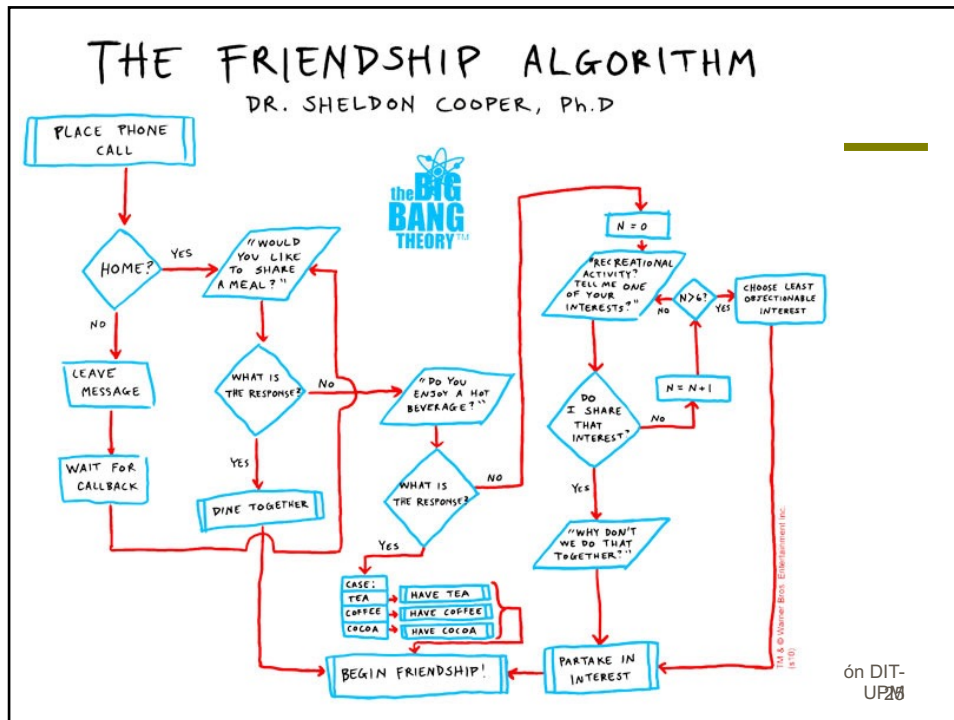
Escribir en Java el *algoritmo de la amistad de Sheldon*. Como implementamos la notación de diagramas de flujos de datos:

Podemos representar las operaciones de entrada/salida (∇), proceso ($\square$) y proceso predefinido ($\square$) mediante llamadas a métodos privados que supondremos que están implementados.

Las condiciones ($\diamond$) complejas las podemos implementar como llamadas a métodos privados implementadas que devuelven valores lógicos.

<https://youtu.be/k0xgjUhEG3U>

● 24



● 25

Contenidos

1. Repaso de herramientas

2. Repaso de programación

2.1 Tipos simples de datos y expresiones

2.2 Sentencias

2.3 Clases y Objetos

2.4 Interfaces y Herencia

2.5 Excepciones

2.6 Bibliotecas

● 26

Clases

- Las clases soportan varios conceptos:
 - Definición de tipos de datos
 - Definen los datos (campos) y comportamiento (métodos) para representar conceptos tratados del programa
 - Espacios de nombres
 - La clase define un ámbito de nombres reconocibles en todo su espacio
 - Encapsulamientos
 - Las clases permiten agrupar datos y comportamientos relacionados

Algoritmos y Estructuras de datos DIT-UPM

● 27

Campos

- Datos primitivos y referencias para los objetos de una clase
- Se declaran en cualquier punto dentro de una clase **menos dentro de un método**
- Sintaxis:
 - Tipo nombre;
 - Tipo nombre1,nombre2,... nombreN;
 - int i;
 - Punto origen, destino;
 - String nombre;
 - Persona propietario, conyugeDelPropietario;

Algoritmos y Estructuras de datos DIT-UPM

● 28

Métodos

- Cabecera

acceso tipo nombre (parametros ...) excepciones

- Cuerpo

- bloque
- secuencia de sentencias
- declaración de **variables automáticas**
- los **parámetros se comportan como variables**

- Colocación

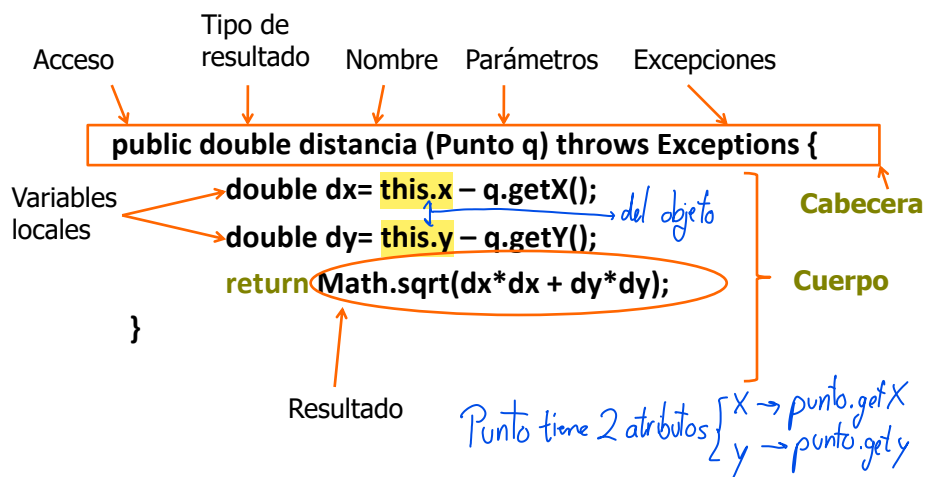
- Dentro de las clases, el orden de los métodos no importa
- **NO** dentro de otro método ni fuera de una clase

Algoritmos y Estructuras de datos DIT-UPM

*acceso que devuelve nombre(parametro){ }
public int suma(int a, int b) { } :-*

● 29

Anatomía



Algoritmos y Estructuras de datos DIT-UPM

● 30

Ejemplos Campos

Algoritmos y Estructuras de datos DIT-UPM

● 31

Ámbitos

- El cuerpo de una clase determina un ámbito
 - Métodos y campos que declara son visibles en toda la clase
- Un método (incluido constructores) determina otro ámbito
 - Los parámetros son visibles en el ámbito del método
- Un bloque de código es otro ámbito
 - Bloques de código
 - Cuerpo de métodos y constructores
 - Cuerpos de sentencias
 - La declaración de variables es visible en las sentencias que siguen a la declaración

Algoritmos y Estructuras de datos DIT-UPM

● 32

Ejemplos Ambitos

Algoritmos y Estructuras de datos DIT-UPM

● 33

Relaciones entre Clases

- Las clases son un mecanismo complejo para soportar la reutilización del software (no reinventar la rueda)
- Las clases integran funcionalidades para tratar al tipo de concepto que soportan
- Una clase compleja se puede construir a partir de clases simples
- Tipos de relaciones entre clases
 - Uso
 - Composición
 - Agregación
 - Dependencia
- En Java las implementamos todas mediante `import`, referencias a clases y referencias a objetos

Programación DIT-UPM

● 34

Contenidos

1. Repaso de herramientas

2. Repaso de programación

2.1 Tipos simples de datos y expresiones

2.2 Sentencias

2.3 Clases y Objetos

2.4 Interfaces y Herencia

2.5 Excepciones

2.6 Bibliotecas

● 35

Interfaces

- Contiene las **signaturas de los métodos comunes que definen un servicio**
- Especificación: **clase vacía**, no puede contener **constructor**
- Clase incompleta (abstracta) que sólo contiene
 - **cabeceras de métodos públicos (no estáticos)**
 - **constantes**
- Una **interfaz es un tipo**
 - **se pueden declarar referencias**
 - Que podrán apuntar a objetos de cualquier clase que implemente a la interfaz
- Una **clase puede implementar varios interfaces**
 - Si no da cuerpo a todos los métodos: **clase abstracta-incompleta**

● 36

Interfaz solo contiene
signatura, sin declarar

```
public interface Cansado {  
    public void desconsolar;  
}
```

```
public class Empleado  
    implements Cansado {  
    @Override  
    public void desconsolar() {  
    }  
}
```

Ejemplos

```
interface Coordenada {
    double getX ();
    double getY ();
    double distancia (Coordenada q);
}
```

Coordenada es clase padre

```
class Cartesiana
    implements Coordenada {
    private double x, y;
    Cartesiana (double x, double y) {
        this.x=x; this.y=y;
    }
    double getX () { return x; }
    double getY () { return y; }
    double distancia (Coordenada q){
        double dx=q.getX()-this.getX();
        double dy=q.getY()-this.getY();
        return Math.sqrt(dx*dx+dy*dy);
    }
}
```

CONSTRUCTOR DE OBJETO

```
class Polar
    implements Coordenada {
    private double r, a;
    Polar (double r, double a) {
        this.r=r; this.a=a;
    }
    double getX() { return r*Math.cos(a); }
    double getY() { return r*Math.sin(a); }
    double distancia (Coordenada q){
        double dx=q.getX()-this.getX();
        double dy=q.getY()-this.getY();
        return Math.sqrt(dx*dx+dy*dy);
    }
}
```

CONSTRUCTOR

Algoritmos y Estructuras de datos DIT-UPM

● 37

//this hace referencia al objeto de su propia clase

Extensión-herencia

● Herencia: extensión

- Se parte de una clase que ya está hecha y probada
- Se crea una **clase nueva que añade atributos y/o métodos**
- Debe existir alguna relación lógica entre la clase ya hecha (clase base) y la nueva (clase derivada): **es-un**

● El uso de la extensión permite

- **Reutilizar las clases hechas** como parte de nuevas clases
- Reducir tiempo y esfuerzo de desarrollo
- **Aplicar el polimorfismo entre clases** que se parecen porque hay extensión entre ellas
- **Cambiar el funcionamiento de algún método** de la clase base

Algoritmos y Estructuras de datos DIT-UPM

● 38

@Override → sobrescribir métodos heredados

//static: atributo no depende del objeto, pertenece a la clase. Se "comparte" entre objetos
 public: En cualquier lugar
 protected: El mismo paquete, o cualquier subclase(herencia) (incluso en otro paquete)
 (sin palabra clave): Solo el mismo paquete — package-private
 private: Solo la misma clase

Ejemplo

```
class Pixel extends Punto {
    private String color; → atributo especial
    public void setColor (String c) {
        color = c;
    }
    public String getColor() {
        return color;
    }
    public void reset () {
        super.reset();
        color = "transparent";
    }
}
```

Punto

```
x : double
y : double

getX() : double
getY() : double
setX(x : double) : void
setY(y : double) : void
modulo() : double
reset() : void
```

Pixel

```
color : String

getColor() : String
setColor(color : String) : void
reset() : void
```

Relación de extensión "es-un"

Algoritmos y Estructuras de datos DIT-UPM

● 39

Ejemplos Hijo, Padre y Abuelo

```
public class Persona {
    //Atributos- característica de la clase y que la define
    String nombre;
    static int numPersonas;

    //CONSTRUCTOR- nomenclatura especifica
    public Persona(String nombre) {
        this.nombre = nombre;
        ++numPersonas; //Atributos por definicion pertenecen a un objeto
    }

    public void comer() {
        System.out.println("Estoy comiendo");
    }
}
```

```
public class Empleado extends Persona implements Cansado{
    public Empleado(String nombre) {
        super(nombre); ← atributo heredad super(atributo);
    }

    public int calcularSalario() {
        return 0;
    }

    @Override //debe implementar el metodo implementado por la interfaz Cansado
    public void descansar() {
        System.out.println("zzzzz");
    }
}
```

Algoritmos y Estructuras de datos DIT-UPM

● 40

```
public EmpleadoFjo(String nombre, int horas) {
    super(nombre);
    this.horas = horas;
}

@Override
public int calcularSalario() { //sin override no sobrescribe
    return 20*horas;
}
```